
飞书 AI 校园挑战赛 · 开营分享

飞书 AI-friendly 分享

不只是在飞书里使用 AI
而是让 AI 更好地使用飞书



黄梦轩

飞书 · 研发工程师

Lark App-Core Services

大三开始在飞书实习
校招加入飞书至今,约 6 年

当前负责

飞书面向 AI 的 开放性友好

- 飞书 MCP
- 飞书 CLI
- 与 OpenClaw、Hermes 等 Agent 集成

今天分享的,就是最近团队在做的事。

为什么飞书不只是在做"飞书里的 AI"?

A · 现有视角

在飞书里使用 AI

在飞书里多了一个 AI 聊天入口，本质上还是加功能



B · 本次真正要讨论的

让 AI 更好地使用飞书

让 AI 能理解飞书、调用飞书能力、真的参与协作

这场分享真正要讨论的，不只是"飞书里的 AI"，而是"AI 如何更好地使用飞书"。

什么叫 AI-friendly?

定义

AI-friendly = 让 Agent 更容易发现、理解、调用、组合

01

可发现

Agent 能知道
系统里有哪些能力可用

02

可理解

Agent 能看懂
能力的语义和参数

03

可执行

Agent 能稳定调用
少踩坑、少重试

04

可编排

Agent 能把多个能力
串成完整流程

不是多一个 AI 按钮，而是一整套面向 Agent 的系统设计。

第 1 章 · 接下来聊聊

先从 MCP 说起

大家都在讲的 MCP——它到底解决了什么？又留下了什么？

MCP：为什么大家都在讲它？

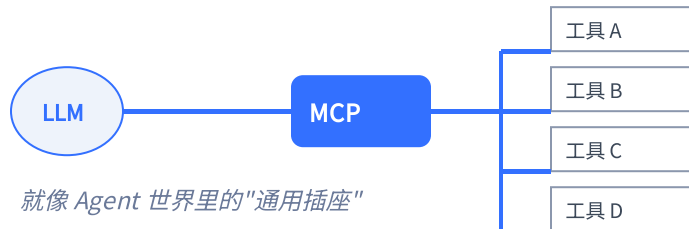
过去

每个工具各有各的接法



现在

MCP 提供更统一的工具接法



MCP 是第一个专为 AI Agent 设计、并被广泛采纳的工具接入标准。

MCP：工具定义先全量进 context，再按需调用

Model Context Protocol — 启动阶段和运行阶段分开看

① 启动阶段

Server 把所有 tool 定义一次性注入模型 context



② 运行阶段

模型从 context 里选中某个 tool，发起调用、等待返回



MCP 通过标准化解决了“怎么接”这件事。

两类问题：定义进 context, 数据也进 context

MCP 架构下，工具定义 + 所有中间数据，都必须经过模型的 context

类别 A · 工具定义必须预加载

例 1

单个业务域就可能吃掉 18k tokens

飞书云文档 MCP 的 tool 定义 ~18k; 30+ 业务域全量暴露就会破 200k, 远超 context window。

例 2

Schema 越复杂、膨胀越厉害

嵌套参数、枚举、条件必填、跨 tool 复用的 schema——工具数增长时，描述开销是超线性的。

类别 B · 过程数据都经过 context

例 3

返回数据必须进 context, 大结果就装不下

多维表格导出一张就能吃掉整个窗口；MCP 场景下只能截行、截字段、省嵌套，用降低完整性换空间。

例 4

过程数据不能落盘、不能本地处理

无法先把数据保存到本地、再用脚本 / Python 处理——所有中间结果都必须过模型，context 就是唯一的工作台。

MCP 把模型的 context 变成了唯一通道——定义、参数、数据、中间结果，全都挤在这里。

第 2 章 · 接下来聊聊

为什么飞书又要做 CLI

从 MCP 的代价出发，看另一条更适合飞书的路

CLI:用命令给电脑下指令

Command Line Interface,命令行界面 — 比图形界面老得多，也简洁得多

在终端里长这样

```
$ ls
reports/ index.md README.md

$ cd reports/
$ wc -lsales.csv
1042 sales.csv

$ grep ACME sales.csv | head -3
2026-02-08, ACME, 91200
2026-03-15, ACME, 78900
```

特征 01

文本输入、文本输出

你打字，它回文字。没有按钮，没有弹窗，没有鼠标。

特征 02

每个命令干一件事，可以管道组合

ls、grep、wc... 小工具用 | 拼起来，威力放大。

特征 03

脚本可复现、可自动化

同一条命令序列，别人、明天、机器都能重跑。

CLI 看起来朴素，但它是计算机诞生以来最稳的人机接口——也正是 AI 最熟悉的一种。

从 CLI 到 GUI,又从 GUI 回到 CLI

不是复古，是因为界面的主要用户从人变成了 AI

1970s — 起点

CLI 一统天下

主要用户：程序员 / 运维

- 命令行是唯一的交互方式
- 精确、可脚本、可组合
- 学习成本高，对普通人不友好

1990s — 普及

GUI 让计算普及给所有人

主要用户：每一个用计算机的人

- 图标、窗口、鼠标拖拽
- 不用记命令，看着就能点
- 代价：动作无法被机器精确引用

2020s — 反转

AI 来了，CLI 再次成为接口

主要用户：Agent

- AI 读/写文本，不会点鼠标
 - CLI 的稳定、可组合、可脚本
- 正是 Agent 需要的能力

关键转变：界面的主要用户不再只是人——当一端是 AI, 文本接口比图形接口更自然。

GUI 是给人用的；CLI 现在也是给 AI 用的。

AI 怎么把 CLI 用起来

不需要专门的协议，Agent 就像程序员一样：在 shell 里敲、看返回、继续敲

1

通过 subprocess 执行命令

Agent 像在终端里一样调用 shell,把命令字符串交给 bash/zsh 执行。不需要专用 MCP 协议。

2

用 --help 自己探索能力

不熟的命令就 `cmd --help`、`cmd sub --help`,一层层看下去。工具定义不用预先塞进 context。

3

拿 stdout 当结果

输出是结构化文本(JSON 或行)。大结果可以写到文件、用 jq 过滤、只把需要的片段放回 context。

4

失败就读错误、改命令、重试

命令出错会在 stderr 里说清楚问题。Agent 看错误、修参数、再来一次——跟人 debug 一样。

Agent:我帮你查明天的会议安排

```
$ lark calendar agenda --date tomorrow --format json
```

```
[{"time":"10:00","title":"周会"}, {"time":"14:30","title":"客户 review"}]
```

Agent 用 CLI 的方式，跟程序员用 CLI 完全一样——这是 MCP 做不到的自然。

光有 CLI 还不够：Agent 需要说明书

--help 告诉 Agent 有什么；Skills 告诉 Agent 什么时候用、怎么组合、要避什么坑

原则 01

按需加载：相关了才读，不预占 context

Skills 按业务域组织成 Markdown 文档。Agent 做日历任务才读 calendar/SKILL.md, 做文档才读 doc/SKILL.md。Context 空间只留给当前任务真正需要的信息。

原则 02

渐进式披露：从总到分，一层层展开

先读顶层概览知道有什么能力；要用到时再展开具体操作的细则、踩坑提示、示例。复杂度按需释放，不会一上来就被细节淹没。

SKILL.md 里通常写什么

```
# calendar/SKILL.md
## 何时使用 → 约会议、查日程、看谁有空
## 常用操作 → lark calendar events create / freebusy ...
## 坑 → 时间必须 RFC3339; attendees 要 open_id 不是邮箱
```

CLI 给能力，Skills 给使用方式——组合起来，才能让 Agent 稳定地做对事。

查看“明天的日程”：两种走法

对通用 Agent 来说，CLI 解决执行，Skills 解决理解

方式 A · 自己摸索

- 1 查 API 文档
- 2 找接口
- 3 拼时间参数
- 4 处理返回格式
- 5 反复重试

方式 B · 加载 Skill

- 1 加载 Skill
- 2 直接执行命令

```
$ calendar +agenda --tomorrow
```

CLI 解决执行问题，Skills 解决理解问题，两者结合更适合广覆盖场景。

CLI 是如何解决 MCP 遇到的问题的

把第 7 页的两类四例，逐条回应——CLI + Skills 天然避开了全量预加载

例 1

单域工具定义就吃掉 18k tokens

CLI + Skills

不预加载，Agent 需要时查 --help / schema,用完即扔

例 2

Schema 越复杂，描述膨胀越厉害

CLI + Skills

Skills 按需加载，只读当前任务相关业务域的文档

例 3

返回数据必须进 context,大结果装不下

CLI + Skills

输出走 stdout,可 jq 提取、写文件，不必全进 context

例 4

过程数据不能落盘、不能本地处理

CLI + Skills

过程数据落本地，用 Python 等处理，只把结果给模型

MCP 是补充层，不是对立面。CLI + Skills 在飞书这种广 API、多业务域场景下更适合通用 Agent。

飞书 CLI:面向 AI Agent 的统一接入层

不是多一个命令行工具，而是在 OAPI 之上补上 Agent 需要的那层适配

20+

个业务域

IM · 文档 · 日历 · 多维表格
任务 · 云盘 · 审批 ...

300+

条 Commands

每条命令 = 一个能力
Agent 能直接调用

20+

套 AI Agent Skills

按业务域配套的使用说明
告诉 Agent 怎么用、避什么坑

接入友好 · 找得到、看得懂 统一入口 + 结构化 schema + Skills 文档——让 AI 知道该调什么、怎么调

执行友好 · 调得通、调得稳 内置认证 / 分页 / 文件 IO; Meta 自动生成，手动封装高频场景——多层协同提端到端成功率

重点不是“命令行”三个字，而是“统一接入层”——让飞书能力变成 Agent 可以稳定消费的操作面。

lark-cli 的四层架构 + Skills

越往上 → 越场景化、越封装；越往下 → 越通用、越原子、覆盖越广



SKILLS · 横贯四层

告诉 Agent 什么场景去哪一层、怎么调、避什么坑——把分散的能力翻译成可靠的工作流。

分层意味着：越上层投入越多、覆盖越少、成功率越高；越下层自动生成、覆盖全部、留给 Agent 自由组合。

第 3 章 · 最后收一下

回到一件更大的事

Agent 生态正在涌入飞书——飞书正在成为面向 AI 的协作平台

以 OpenClaw 为例

OpenClaw 是当前最为流行的通用 AI Agent 之一。它通过飞书官方插件,已经深度接入了飞书——可以以用户身份看文档、发消息、约会议、管理任务。

接下来两页,看它怎么和飞书结合

- ① 视角一 · 飞书作为 channel —— 用户在飞书里和 AI 协作
- ② 视角二 · 飞书作为工具 —— AI 动手操作飞书,帮人做事

OpenClaw 是一个样本,背后是一类正在扩大的趋势——Hermes 等同类 Agent 也在陆续接入。

飞书作为 channel：用户和 Agent 在飞书里协作

OpenClaw 直接进驻飞书——让 AI 对话发生在你本来就工作的地方

#产品组 · @OpenClaw

你

帮我约明天下午和张三的一小时会

OpenClaw

已创建：明天 14:00-15:00，邀请已发送

你

把这周的 OKR 同步到公司多维表格

OpenClaw

已同步 3 条 OKR，查看多维表格 →

1

对话就是入口

用户不用切应用、不装客户端 — @一下机器人就开始用

2

上下文在协作流里

群消息、线程、文档引用，Agent 直接读到真实语境

3

流式卡片 + 交互控件

回答边生成边显示，结果可以一键确认/修改

把 AI 对话放回协作现场——它看到的上下文,和你看到的一样。

飞书作为工具：Agent 动手操作飞书

OpenClaw 通过 lark-cli 读文档、发消息、改表格,甚至以自己的身份登录飞书



Agent 像调用函数一样调 lark-cli,不关心 HTTP/鉴权/分页/错误处理

帮你动手做事

帮你整理文档 / 改表格 / 发消息 / 约会议,不只是给你文字建议

以自己身份上飞书

Agent 可以拥有独立飞书账号,作为团队成员参与协作

AI 不只能"聊",还能"做"——当 Agent 能自己登录飞书,协作就多了一类新成员。

两个视角合起来:人在飞书里和 AI 协作,AI 也在飞书里和人协作。

当各种 Agent × 各种场景都能接入飞书

OpenClaw 只是开始。lark-cli 作为统一接入层，让任何 Agent 都能消费飞书的协作能力。

各种 Agent

- Codex
- Trae
- OpenClaw
- Hermes
- …你自己写的 Agent

统一接入层

lark-cli

+ MCP / Skills

各种场景

- 日程 · 会议 · 忙闲
- 文档 · 多维表格
- 消息 · 群协作
- 任务 · 审批
- …更多业务场景

这，就是我们要做的——真正 AI-friendly 的工作平台。

飞书 CLI 已经开源

你可以直接用它,也可以把自己的 Agent 接上来——一起把飞书做成 AI 能用的协作平台。

GitHub

github.com/larksuite/cli

场景 1

自己用

装上就能让 Codex、Trae 操作飞书

场景 2

接入自己的 Agent

把 lark-cli 当 tool 层 + 消费 Skills

场景 3

参与贡献

欢迎提 issue、Skill、业务域扩展

你们做的 Agent 也可以直接接上来。

结尾

不只是在飞书里使用 AI
而是让 AI 更好地使用飞书